

Monte Carlo Path Tracer

Computer Graphics — Assignment 1 Report

Georgy Kuznetsov

May 18, 2026

1 Overview

This is report of the first assignment of a Monte Carlo pathtracer (look for slides), additionally I implemented transparent materials (surfaces) and depth-of-field camera. We rendered everything at 512×512 pixels, check for samples under images. All of them are png and saved on this repo. Honestly, I will try to squeeze 4 pages out of it.

2 Lab 1 — Ray–Sphere Intersection, Direct Lighting, and Mirror Surfaces

A ray $\mathbf{r}(t) = \mathbf{O} + t\mathbf{u}$ intersects a sphere of centre $\mathbf{C}$ and radius R when

$$(\mathbf{u}(\mathbf{O} - \mathbf{C}))^2 - (\|\mathbf{O} - \mathbf{C}\|^2 - R^2) \geq 0.$$

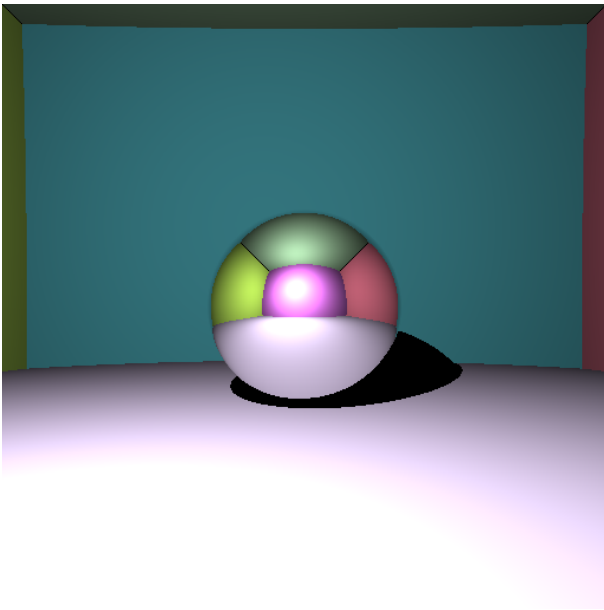
We compute both roots and keep the smallest non-negative one. The surface normal at the hit point $\mathbf{P}$ is $\mathbf{N} = (\mathbf{P} - \mathbf{C})/R$. Then in *Scene* for each object call it intersect function (for now just spheres) and we keep the closest hit.

I was stuck here and asked for help Julia Korytkowska. Parts of this section were developed in collaboration with Julia Korytkowska; some similarities in the code may reflect that discussion.

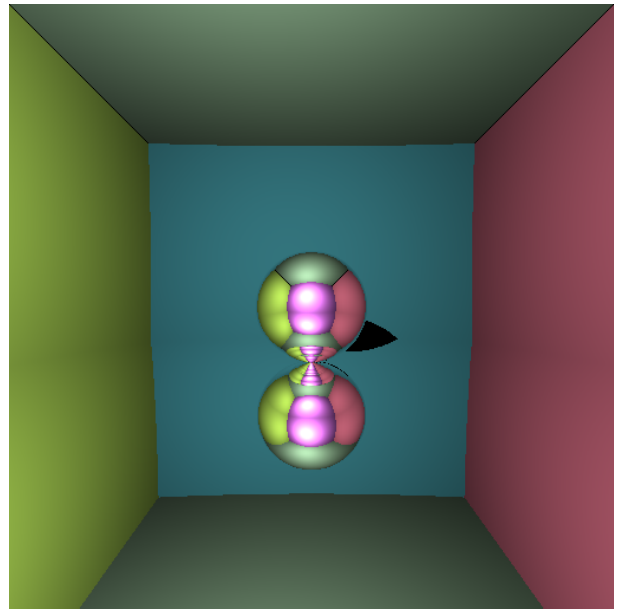
Then we compute *Scene* *getColor()* function. If we hit the ray, then we try to either cast it further (when transparent) or compute the light contribution. As well we cast a shadow ray to the light source to check if the point is in shadow. (with some jitter due to floats).

Transparent - we check if the ray is entering or exiting the object by checking sign of $\text{dot}(\text{ray.u}, \mathbf{N})$. Then we compute cosines and sines with Snell's law. And then return recursively with refracted ray. This piece of code was inspired by Chapter 9: Refraction, Ray Tracing in One Weekend, as well as comments were crafted with Claude Code.

When rendering mirrors we flip the ray when it hits the surface with: $\mathbf{u}' = \mathbf{u} - 2(\mathbf{u} \cdot \mathbf{N})\mathbf{N}$ and the path tracer is called recursively with depth incremented. The maximum bounce count is 5.



(a) Mirror sphere with shadows only (1 spp, no indirect lighting). Render time 1sec



(b) Mirror sphere with mirror floor (max bounce 15). Render time 2sec.

Figure 1: Lab 1 results

3 Lab 2 — Indirect Lighting, Antialiasing, and Depth of Field

Indirect lighting - we look back at *getColor()* function. No instead of direct ray hitting surface we assume that it is diffuse and we sample with cosine-weighted hemisphere. Casted with some jitter, we send a recursion with :

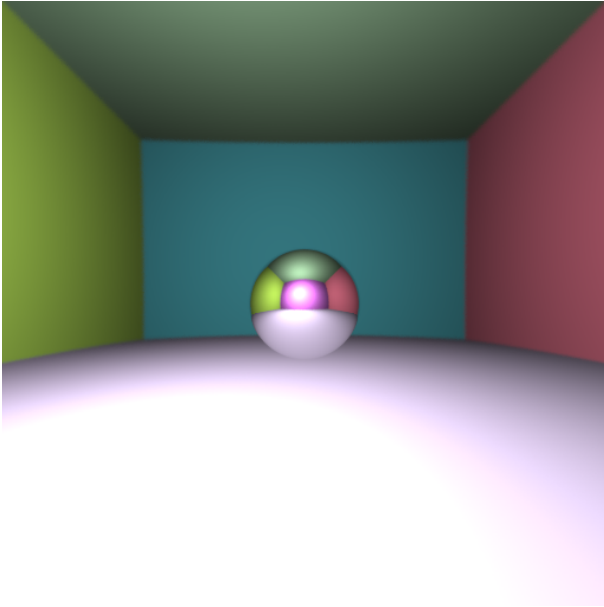
```
indirect_light = objects[object_id]->albedo *
    getColor(new_ray, recursion_depth + 1);
```

This way we get Monte-Carlo for indirect lighting. No Russian roulette is used; recursion terminates at depth 5. Parts of this section were developed in collaboration with Julia Korytkowska; some similarities in the code may reflect that discussion.

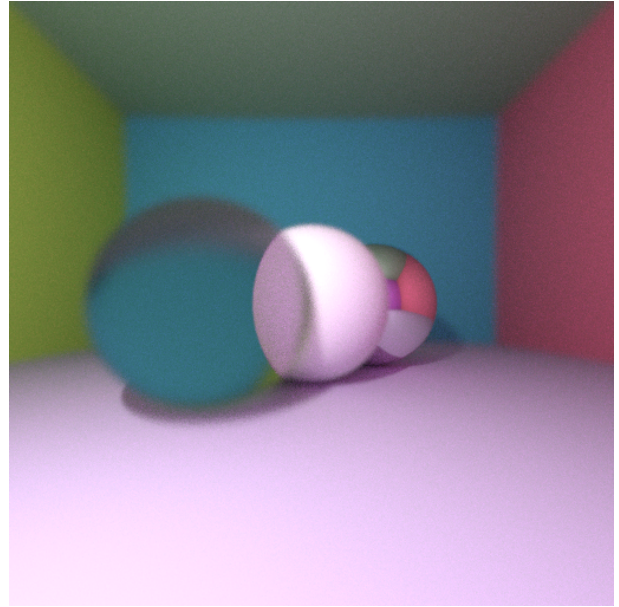
Each pixel is averaged over 32 samples.

Antialiasing. Per pixel we apply some jitter to the ray origin to get a smoother edges. We draw from Gaussian with $\sigma = 0.5$ pixels using the Box-Muller transform. Here there were some problems with fopenmp on MacOS (we talked with TA about it) and decided to keep just the first engine for rendering, so the render times are a bit higher than expected. Parts of this section were developed in collaboration with Julia Korytkowska; some similarities in the code may reflect that discussion.

Depth of Field. We implemented a thin lense camera with simulating a focal point and aperture. focall point is at camera center plus focal distance (40 units) in the forward direction. we sample a Gaussian with some deviation of aperture size to get a new ray origin then instead of shooting the ray from tje camera center we cast it from the focal point.



(a) Indirect lighting, definitely can see light from the mirror being cast onto walls and floor. 32 spp, 512×512, 20s on one thread of macos (omp fails).



(b) Depth-of-field effect ($f = 40$, $a = 3$)

Figure 2: Lab 2 results

4 Labs 3 & 4 — Triangle Mesh Intersection and BVH

4.1 Triangle Intersection

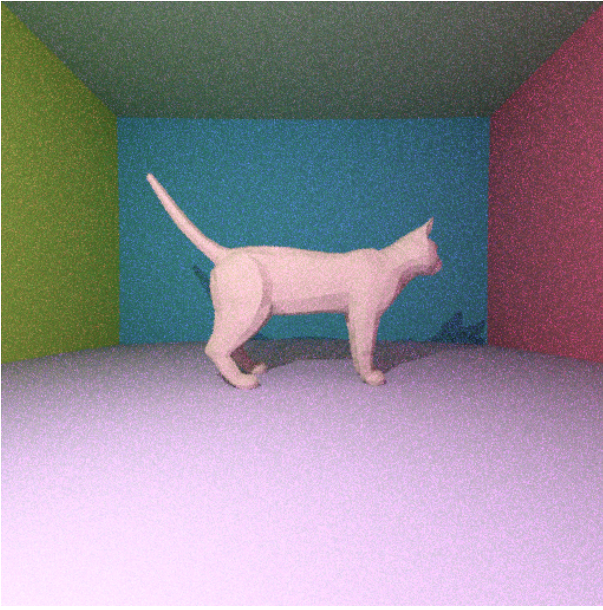
We used the Moller-Trumbore algorithm from the lecture notes. Given vertices v_0, v_1, v_2 and edges $e_1 = v_1 - v_0$, $e_2 = v_2 - v_0$, we can compute ray parameter t and coordinates α, β, γ as follows:

$$\beta = \frac{\mathbf{e}_2 \cdot [(\mathbf{v}_0 - \mathbf{O}) \times \mathbf{u}]}{\mathbf{u} \cdot (\mathbf{e}_1 \times \mathbf{e}_2)}, \quad \gamma = \frac{-\mathbf{e}_1 \cdot [(\mathbf{v}_0 - \mathbf{O}) \times \mathbf{u}]}{\mathbf{u} \cdot (\mathbf{e}_1 \times \mathbf{e}_2)}, \quad \alpha = 1 - \beta - \gamma, \quad t = \frac{(\mathbf{v}_0 - \mathbf{O}) \cdot (\mathbf{e}_1 \times \mathbf{e}_2)}{\mathbf{u} \cdot (\mathbf{e}_1 \times \mathbf{e}_2)}.$$

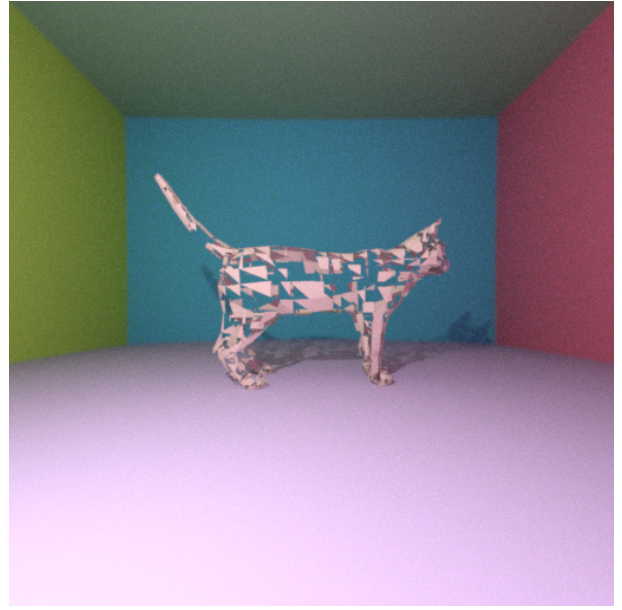
We hit the triangle if $t > 0$ and $\alpha, \beta, \gamma \geq 0$. The normal is just cross product of two edges $e_1 \times e_2$ and flipped to face the incoming ray. Parts of this section were developed in collaboration with Julia Korytkowska and Alisa Karlowa; some similarities in the code may reflect that discussion.

Bounding Volume Hierarchy (BVH). Oooh I spend a lot of time on this one...

Sp first we do a simple BVH implementation with just computignmax bounding box over the whole object and do ray-plane intersections with the bounding box. We build it recursively : we split triangles into halves and compute triangle ranges. (somth like quicksort - $O(\log n)$). Then we upgraded it to building binary trees. We do the same, we split triangles along the largest axis of current bounding box. Then recursively build left and right subtrees. Until we have 5 or less triangles - then a leaf. Then we want to get if the ray hits the bounding box - each box has simple ray-plane intersection test. So we start from the root, check if this simple intersection is hit, if not - we skip the whole subtree. If it is hit, we check the left and right child. If it is leaf - we do triangle intersection test with all triangles in the leaf and return the closest one. We use stacks, to avoid unnecessary recursion.



(a) Cat mesh (3954 triangles) 32 spp, no omp, 30s.



(b) One of the intermediate debug images (just funny picture)

Figure 3: Labs 3 & 4 results

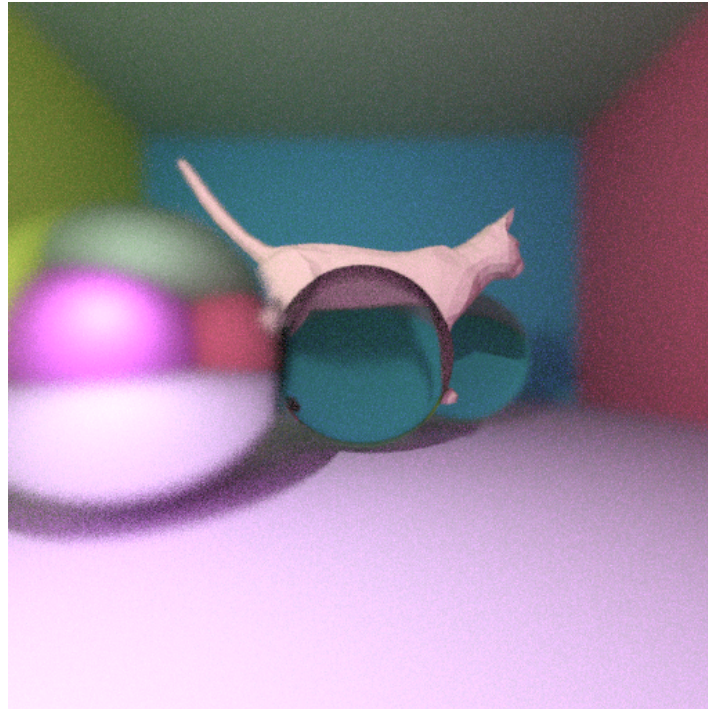


Figure 4: Final scene combining all features: cat mesh with BVH, left mirror sphere, right transparent glass sphere (IOR 1.5), Gaussian antialiasing ($\sigma = 0.5$ px), and depth-of-field blur ($f = 40$, aperture $a = 1$). 32 spp, 5 bounces, 512×512, single-threaded MacBook Air M1: **41 s**.